

Závěrečná dokumentace

Petri Net Analyzer

Architektura, testování a konfigurační management

Verze 1.0

8. května 2026

Položka	Hodnota
Projekt	Petri Net Analyzer
Dokument	Závěrečná dokumentace
Verze	1.0
Autor	David Šigut
Repozitář	https://github.com/seracxd/PetriNet

Obsah

1	Úvod	2
2	Architektura aplikace	2
2.1	Architektonický styl	2
2.2	Schéma architektury	2
2.3	Schéma rozmístění (deployment)	3
2.4	Použité technologie a nástroje	4
3	Testování	5
3.1	Strategie testování	5
3.2	Typy testů	5
3.3	Použité nástroje	6
3.4	Pokrytí kódu	6
3.4.1	Statistika testů	6
3.4.2	Naměřené pokrytí kódu	7
3.4.3	Pokrytí jednotlivých tříd v PetriEditor.Analysis	8
3.4.4	Reprodukce výsledků	9
3.5	Ukázky testů	9
3.5.1	Jednotkový test — invariantní vlastnost simulátoru	9
3.5.2	Theory test — sémantika reset hran	9
3.5.3	Cross-engine konzistence	10
3.6	Strategie testování — kdy a jak	10
4	Konfigurační management	11
4.1	Verzovací systém	11
4.1.1	Struktura repozitáře	11
4.1.2	Větvení a workflow	11
4.2	Build systém	11
4.2.1	Klíčové příkazy	12
4.2.2	NuGet závislosti	12
4.3	Konfigurace aplikace	12
4.3.1	appsettings.json (server)	12
4.3.2	Environment variables	13
4.3.3	Klientská konfigurace	13
4.4	Dockerizace	13
4.5	CI/CD	14
5	Bezpečnostní opatření a omezení	15
6	Závěr	16

1 Úvod

Tento dokument popisuje výsledný stav projektu **Petri Net Analyzer** — interaktivního editoru, simulátoru a formálního analyzátoru Petriho sítí provozovaného jako webová aplikace. Cílem dokumentace je poskytnout úplný pohled na tři hlavní oblasti projektu:

- **Architekturu aplikace** (kapitola 2) — zvolený architektonický styl, schéma komponent a použité technologie.
- **Testování** (kapitola 3) — typy testů, použité nástroje, pokrytí kódu a strategii ověřování.
- **Konfigurační management** (kapitola 4) — práce s verzovacím systémem, build systém, konfigurace aplikace a CI/CD.

Detailní popis návrhu (4+1 view, sekvenční diagramy, modely tříd) je součástí samostatného dokumentu *Software Architecture Document*, který tvoří přílohu k tomuto materiálu.

2 Architektura aplikace

2.1 Architektonický styl

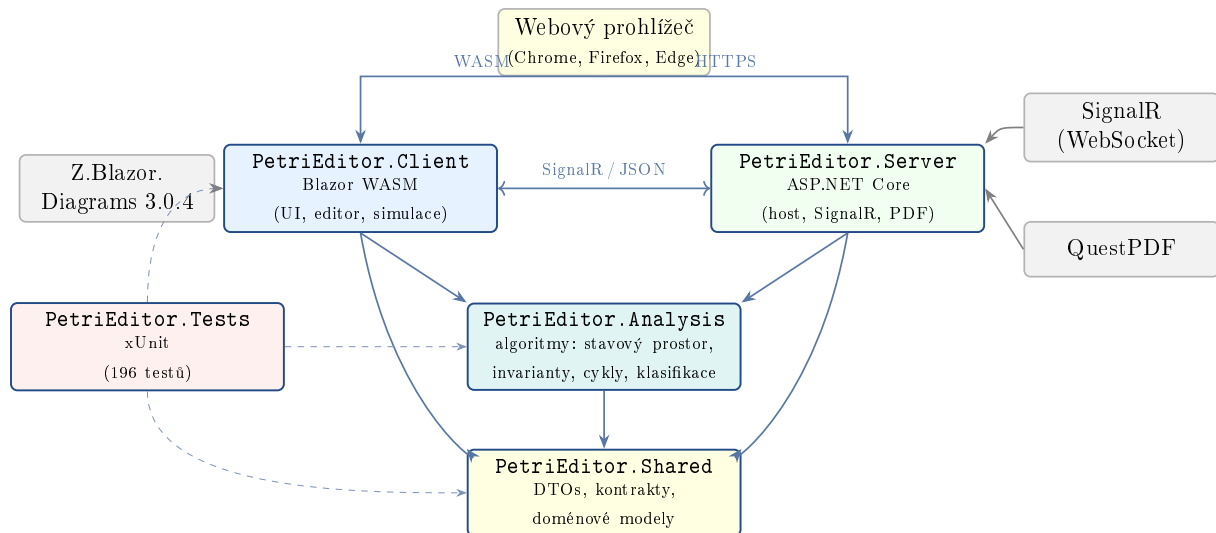
Aplikace je realizována jako **vícevrstvý klient-server systém** s tenkým serverem. Konkrétně se jedná o kombinaci dvou stylů:

- **Modulární monolit** z pohledu nasazení — celý systém (server, sdílené knihovny, klient) je distribuován jako jediný ASP.NET Core proces běžící na serveru. Z hlediska modularity je rozdělen do pěti logicky oddělených .NET projektů, jejichž závislosti tvoří orientovaný acyklický graf.
- **Klient-server s tlustým klientem** z pohledu běhu — klientská aplikace je doručena prohlížeči jako Blazor WebAssembly modul a veškerá interaktivní práce (editace diagramu, simulace, undo/redo) probíhá lokálně v prohlížeči. Server slouží primárně k 1) prvotnímu doručení statických webových assetů a 2) výpočetně náročné formální analýze přenášené přes SignalR.

Volba tohoto stylu vychází z požadavku na minimalizaci síťové komunikace během běžné editace (offline-friendly), ale zároveň možnosti odlehčit slabšímu klientskému zařízení od těžkých výpočtů (analýza stavového prostoru sítí s desítkami tisíc stavů). Pro tento dvojí provoz je využita technologie **Blazor Auto** (interactive WebAssembly + interactive Server render mode).

2.2 Schéma architektury

Architektura je rozdělena do pěti samostatných .NET projektů (viz solution file `PetriNet.slnx`). Vrstvení a závislosti ukazuje obrázek 1.



Obrázek 1: Diagram komponent — pět .NET projektů a jejich vztahy k externím knihovnám a běhovému prostředí. Plné šipky = závislost na sestavení (ProjectReference); čárkované šipky = reference pouze v testech.

Popis projektů

PetriEditor.Shared Sdílené doménové modely Petriho sítě (Place, Transition, Arc, ArcType), DTO objekty pro SignalR přenos a mapovací rozhraní. Je referenční bod, na který se odkazují všechny ostatní projekty — tvoří *spodní vrstvu* grafu závislostí.

PetriEditor.Analysis Knihovna formální analýzy bez závislosti na UI nebo webovém kontextu. Obsahuje pět nezávislých analytických enginů (stavový prostor / Karp-Miller, P/T-invarianty, klasifikace struktury, Johnsonův algoritmus pro cykly, traps & siphons) a sedm property testů (liveness, boundedness, safety, conservativeness, repetitiveness, deadlock-freedom, reversibility). Lze ji spustit jak v klientovi (in-process), tak na serveru.

PetriEditor.Client Vlastní webová aplikace v Blazor WebAssembly. Vykresluje UI editoru (komponenty Home.razor, AnalysisPanel.razor, GraphTreePanel.razor), spravuje stav diagramu (PetriNetManager), implementuje undo/redo (UndoRedoService, command pattern), interaktivní simulaci (SimulationService) a JS interop s knihovnou Z.Blazor.Diagrams.

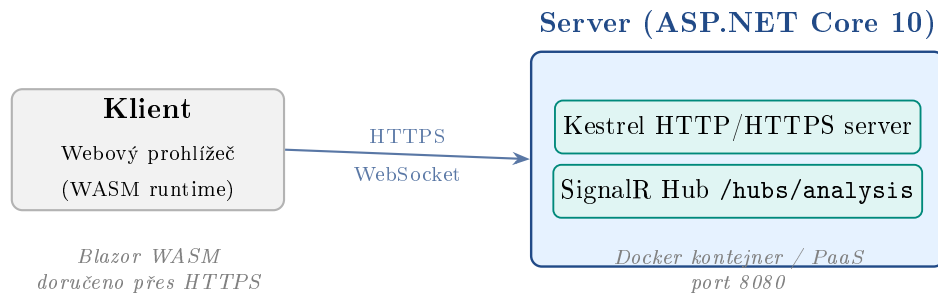
PetriEditor.Server ASP.NET Core 10 host, který doručuje klientské WASM assety, vystavuje SignalR hub /hubs/analysis pro server-side analýzu velkých sítí a generuje PDF exporty výsledků pomocí knihovny QuestPDF. Také obsahuje souborový logger a perzistenci klíčů DataProtection pro stabilitu antiforgery tokenů mezi restarty.

PetriEditor.Tests Unit, integrační a regresní xUnit testy (196 testovacích metod, 4045 řádků) — viz kapitola 3.

2.3 Schéma rozmístění (deployment)

Aplikace má tři typické varianty nasazení (obrázek 2):

1. **Lokální vývoj:** `dotnet run` z projektu `PetriEditor.Server`; klient je servován z paměti a hot-reload je aktivní.
2. **Docker kontejner:** jediný image postavený podle `Dockerfile` (multi-stage build, viz sekce 4.4), deployovatelný na libovolný PaaS (Railway, Azure App Service, ...).
3. **Statické hostování (volitelné):** vzhledem k tomu, že analýza je k dispozici i ve WASM podobě, lze klientskou část publikovat samostatně — pak jsou těžké výpočty omezeny výkonem prohlížeče.



Obrázek 2: Diagram nasazení — tenký server + tlustý WASM klient, obousměrné spojení přes SignalR (WebSocket).

2.4 Použité technologie a nástroje

Technologie	Verze	Účel
.NET	10.0	Cílová platforma všech projektů.
C#	12	Hlavní programovací jazyk.
ASP.NET Core	10.0	Webový server, hosting, antiforgery.
Blazor WebAssembly	10.0	Spuštění .NET kódu v prohlížeči.
Blazor Server	10.0	Interaktivní render-mode pro server-side renderování.
SignalR	9.0.4	Obousměrná komunikace klient–server pro analýzu.
Z.Blazor.Diagrams	3.0.4	Knihovna pro vykreslování diagramů a interaktivitu.
QuestPDF	2024.10	Generování PDF reportů z analýzy.
DataProtection	9.0.3	Trvalé klíče pro antiforgery napříč restarty.
xUnit	2.9.3	Framework pro jednotkové a integrační testy.
coverlet.collector	6.0.4	Sběr metrik pokrytí kódu.
Microsoft.NET.Test.Sdk	17.14.1	Testovací běhový adapter pro <code>dotnet test</code> .
Cytoscape.js (CDN)	3.x	Volitelné renderování coverability grafu (s SRI hashem).
Docker	—	Kontejnerizovaný build a runtime.

Technologie	Verze	Účel
Git + GitHub	—	Verzování, code review, pull requesty.

Klíčové architektonické rozhodnutí Sdílení knihovny `PetriEditor.Analysis` mezi klientem i serverem umožňuje *stejný* algoritmický kód použít v obou prostředích. Pro malé sítě (ojedinělý kontext editace) se analýza spouští přímo v prohlížeči, pro velké sítě je delegována na server přes SignalR s chunked přenosem grafu (200 uzlů/zpráva, viz `AnalysisHub.StreamGraph`).

3 Testování

3.1 Strategie testování

Projekt používá **vrstvenou strategii testování** s důrazem na formální korektnost analytických algoritmů, neboť chybný výsledek formální analýzy by byl pro uživatele zavádějící. Strategie se řídí třemi pravidly:

1. **Algoritmické jádro je 100% testovatelné** — knihovna `PetriEditor.Analysis` nemá žádné UI ani webové závislosti, takže všechny veřejné API jsou pokryty unit testy.
2. **Cross-engine konzistence** — jakékoli dva engine, jejichž výsledky se logicky musí shodovat (např. P-invariant $\Rightarrow$ váha musí být zachována ve všech reachable stavech), jsou ověřeny v `CrossEngineConsistencyTests`.
3. **Klasické literární vzory** — známé sítě (producer-consumer, vzájemné vyloučení, dining philosophers, bounded buffer, token ring) tvoří regresní sadu, která zajistí, že refaktoring algoritmu neovlivní již ověřené výsledky.

3.2 Typy testů

Typ	Příklady v projektu
Jednotkové (unit)	Per-engine test soubory: <code>StateSpaceTests.cs</code> , <code>InvariantAnalysisTests.cs</code> , <code>ClassificationAnalysisTests.cs</code> , <code>CycleDetectionTests.cs</code> , <code>CoverabilityTreeTests.cs</code> , <code>ReachabilityTreeTests.cs</code> , <code>SimulatorTests.cs</code> , <code>TreeLayoutEngineTests.cs</code> , <code>GraphLayoutEngineTests.cs</code> .

Typ	Příklady v projektu
Property-based Theory	/ <code>TheoryTests.cs</code> (parametrizované testy invariantních vlastností: nezávislost reset arc na pořadí, hranice ořezu, deduplikace stavů, vážené hrany, inhibitor threshold, N-loop cykly). <code>xUnit [Theory] + [InlineData]</code> .
Integrační	<code>ClassicalNetsTests.cs</code> — celá pipeline od konstrukce snapshotu po výsledný <code>AnalysisReport</code> pro pět klasických sítí.
Cross-engine regresní	/ <code>CrossEngineConsistencyTests.cs</code> — ověřuje vzájemnou konzistenci výsledků (P-invarianty platí v každém dosažitelném stavu, coverability tree vs. state space, deduplikace cyklů, sémantika trapů).
Coverage gap	<code>CoverageGapTests.cs</code> , <code>DefiniteDeadlockTests.cs</code> , <code>PropertyTestsTests.cs</code> — explicitní testy hraničních případů, ořezu výpočtu a chování pod cancellation tokenem.
End-to-end (manuální)	UI-related testy nejsou automatizovány. Manuální E2E je prováděno během vývoje (drag-and-drop, undo/redo, JS interop) v prohlížečích Chrome a Firefox.

3.3 Použité nástroje

- **xUnit 2.9.3** — primární testovací framework (`[Fact]`, `[Theory]`, `[InlineData]`, `assertce Assert.Equal`, `Assert.Contains`, `Assert.True`).
- **Microsoft.NET.Test.Sdk 17.14.1** — adaptér umožňující spuštění přes `dotnet test` i z Visual Studio / JetBrains Rider.
- **xunit.runner.visualstudio 3.1.4** — VS test runner.
- **coverlet.collector 6.0.4** — sběr code coverage metrik; výstup ve formátu `cobertura.xml` (kompatibilní s ReportGenerator, Coveralls, Codecov).
- **Helpers/** — interní knihovna pro tvorbu testovacích sítí (`NetBuilder` fluent API), aby testy zůstaly čitelné.

3.4 Pokrytí kódu

3.4.1 Statistika testů

V projektu je definováno **196 testovacích metod** (`[Fact]` + `[Theory]`), které dohromady reprezentují **220 testovacích případů** po expanzi parametrizovaných `[Theory]` přes `[InlineData]`. Rozložení podle test-souborů ukazuje tabulka níže.

Test soubor	# Testů	Řádků
ClassicalNetsTests.cs	16	362
ClassificationAnalysisTests.cs	17	280
CoverabilityTreeTests.cs	12	221
CoverageGapTests.cs	11	213
CrossEngineConsistencyTests.cs	13	335
CycleDetectionTests.cs	12	249
DefiniteDeadlockTests.cs	9	201
GraphLayoutEngineTests.cs	11	160
InvariantAnalysisTests.cs	9	187
PropertyTestsTests.cs	18	388
ReachabilityTreeTests.cs	13	243
SimulatorTests.cs	24	401
StateSpaceTests.cs	12	275
TheoryTests.cs	7	286
TreeLayoutEngineTests.cs	12	244
Celkem	196	4 045

3.4.2 Naměřené pokrytí kódu

Pokrytí bylo změřeno spuštěním všech testů s nástrojem `coverlet` (`dotnet test -collect:"XPlat Code Coverage"`) a následným zpracováním `cobertura.xml` pomocí `ReportGenerator`. Souhrnné metriky:

Metrika	Hodnota	Pokryto / celkem
Line coverage	18.1 %	1 485 / 8 199
Branch coverage	15.5 %	711 / 4 559
Method coverage	18.3 %	205 / 1 119
Třídy v reportu	157	—
Soubory zdrojů	72	—

Globální čísla zachycují celý solution včetně UI vrstvy a kontrakt DTO objektů, které nemají žádnou behavior-logiku k testování. Po rozčlenění podle assembly je pohled mnohem informativnější:

Assembly	Line cov.	Účel
PetriEditor.Analysis	81.3 %	algoritmické jádro (cíl testů)
PetriEditor.Shared	26.0 %	doménové modely + DTO kontrakty
PetriEditor.Client	1.7 %	UI / Blazor komponenty

Interpretace:

- **PetriEditor.Analysis (81.3 %)** — *cíl testovacího úsilí*. Algoritmické jádro je systematicky pokryto jednotkovými, theory a regresními testy. Většina jednotlivých tříd dosahuje 90–100 % (viz tabulka per-class níže). Nepokryté zbylé části jsou:

- `Analysis.AnalysisService` (0 %) a `Analysis.AnalysisReport` (0 %) – fasáda volaná z UI; její business-logika je v enginech, které jsou plně pokryty.
- `GraphLayoutBuilder` (0 %) – starý kód nahrazený `GraphLayoutEngine` (98.9 %), ponechán jako fallback.
- **PetriEditor.Shared (26 %)** — pokryty jsou doménové modely využívané analýzou (`PnPlace`, `PnTransition`, `PnArc`, `PetriNetSnapshot` – 83–100 %); nepokryty jsou DTO třídy (`*Dto.cs`) – záměrně, jelikož jde o čistá data-holder POCO bez logiky (pouze auto-generované gettersy a settersy).
- **PetriEditor.Client (1.7 %)** — Blazor UI vrstva. Pokrytí je nízké, protože:
 - Blazor komponenty (`*.razor`) vyžadují rendering host (`bUnit`), který v projektu není zaveden.
 - Drag-and-drop, JS interop a undo/redo workflow se testují manuálně E2E (viz strategie testování).
 - Jediná netriviální pokrytá třída je `TreeLayoutEngine` (99 %), což je čistý algoritmus bez UI závislosti.

3.4.3 Pokrytí jednotlivých tříd v `PetriEditor.Analysis`

Detailní rozpis pokrytí jednotlivých tříd analytického jádra:

Třída	Line cov.
<code>Analysis.Algorithms.BoundednessTest</code>	100 %
<code>Analysis.Algorithms.ConservativenessTest</code>	100 %
<code>Analysis.Algorithms.DeadlockFreeTest</code>	100 %
<code>Analysis.Algorithms.RepetitivenessTest</code>	100 %
<code>Analysis.Algorithms.ReversibilityTest</code>	100 %
<code>Analysis.Algorithms.SafetyTest</code>	100 %
<code>Analysis.Algorithms.LivenessTest</code>	96.6 %
<code>Analysis.Algorithms.FireUtils</code>	97.8 %
<code>Analysis.Algorithms.GraphLayoutEngine</code>	98.9 %
<code>Analysis.Algorithms.CoverabilityTreeBuilder</code>	91.1 %
<code>Analysis.Algorithms.ReachabilityTreeBuilder</code>	94.7 %
<code>Analysis.Engines.ClassificationAnalysis</code>	96.9 %
<code>Analysis.Engines.CyclesAnalysis</code>	95.5 %
<code>Analysis.Engines.InvariantAnalysis</code>	87.6 %
<code>Analysis.Engines.StateSpaceAnalysis</code>	96.0 %
<code>Analysis.Engines.TrapCotrapAnalysis</code>	100 %
<code>Analysis.Simulation.PetriNetSimulator</code>	100 %
<code>Analysis.AnalysisBundle</code>	100 %

Plný HTML report (s navigací do zdrojového kódu, zvýrazněnými nepokrytými řádky a grafy historického trendu) je vygenerován do `docs/coverage/index.html`.

3.4.4 Reprodukce výsledků

Lokální spuštění všech testů s metrikou pokrytí:

```
1 # Vsechny testy
2 dotnet test
3
4 # Včetně sberu pokrytí (coverlet)
5 dotnet test --collect:"XPlat Code Coverage"
6
7 # Volitelně: HTML report z cobertura.xml
8 dotnet tool install -g dotnet-reportgenerator-globaltool
9 reportgenerator -reports:"**/coverage.cobertura.xml" \
10                 -targetdir:"coveragereport" \
11                 -reporttypes:Html
```

Výsledek běhové statistiky: 220/220 testů prochází za ~2 s (release build, Windows 11, .NET 10 SDK). Build je čistý — 0 chyb, 1 varování (NU1510 package-prune, neblokuje).

3.5 Ukázky testů

3.5.1 Jednotkový test — invariantní vlastnost simulátoru

Test ověřuje, že PetriNetSimulator po vyresetování vrátí síť do iniciálního stavu nezávisle na předchozí historii vystřelení:

```
1 [Fact]
2 public void Reset_RestoresInitialMarking()
3 {
4     var net = NetBuilder.Linear(p1: 3, p2: 0);
5     var sim = new PetriNetSimulator(net);
6
7     sim.FireAll(); // vyprazdni p1
8     Assert.Equal(0, sim.Tokens["p1"]);
9
10    sim.Reset();
11    Assert.Equal(3, sim.Tokens["p1"]);
12    Assert.Empty(sim.History);
13 }
```

3.5.2 Theory test — sémantika reset hran

Parametrizovaný test ověřuje, že pořadí výpisu hran (normal vs. reset) nemá vliv na výsledný marking:

```
1 [Theory]
2 [InlineData(ArcOrder.NormalThenReset)]
3 [InlineData(ArcOrder.ResetThenNormal)]
4 public void ResetArc_OrderIndependent(ArcOrder order)
5 {
6     var net = NetBuilder.MixedArcs(order, initialTokens: 5);
7     var sim = new PetriNetSimulator(net);
```

```
8
9     sim.Fire("t1");
10
11     Assert.Equal(0, sim.Tokens["p1"]);    // reset vyhrava
12     Assert.Equal(1, sim.Tokens["p2"]);
13 }
```

3.5.3 Cross-engine konzistence

Test ověřuje, že každý spočítaný P-invariant skutečně platí ve všech dosažitelných stavech (definice P-invariantu: $y \cdot M = y \cdot M_0$):

```
1 [Fact]
2 public void PInvariants_HoldAtEveryReachableMarking()
3 {
4     var net = NetBuilder.MutualExclusion();
5     var bundle = AnalysisService.Run(net.ToSnapshot());
6
7     foreach (var pi in bundle.Invariants.PInvariants)
8     foreach (var marking in bundle.StateSpace.Markings)
9     {
10         var weighted = DotProduct(pi, marking);
11         var initial = DotProduct(pi, bundle.StateSpace.Initial);
12         Assert.Equal(initial, weighted);
13     }
14 }
```

3.6 Strategie testování — kdy a jak

- **Při vývoji:** test-driven přístup pro čisté algoritmy (state space, invariants); nejprve napsán test podle definice z literatury (Murata 1989), poté implementace.
- **Před commit:** `dotnet test` musí projít s 0 selháními; `dotnet build /p:TreatWarningsAsErrors=true` se používá jako lint.
- **Při bug fixu:** je-li to možné, nejprve přidán failing test reprodukcující bug, teprve pak oprava (regresní test). Příklad: `TheoryTests.cs` obsahuje regresní test pro dříve špatné chování reset hran v závislosti na pořadí výpisu.
- **Při refaktoringu velkých částí:** cross-engine testy jsou primární bezpečnostní síť; pokud byly před refaktorem zelené a zůstávají zelené po refaktoru, lze refaktor považovat za sémanticky neutrální.
- **Manuální E2E testování:** prováděno v prohlížeči podle scénářů popsaných v Software Architecture Document (drag-and-drop arc, undo/redo deep stack, simulace s time-travel, dock/float panelu, JS interop pro klávesové zkratky).

4 Konfigurační management

4.1 Verzovací systém

Projekt používá **Git** jako verzovací systém s repozitářem hostovaným na GitHubu (<https://github.com/seracxd/PetriNet>).

4.1.1 Struktura repozitáře

```

1 PetriNet/
2 |-- .github/
3 |   '-- workflows/                # Pripraveno pro CI (zatim prazdne)
4 |-- .gitattributes                # Normalizace EOL a binarnich souboru
5 |-- .gitignore                   # Standardni VS / dotnet ignore
6 |-- Dockerfile                   # Multi-stage build & deploy
7 |-- PetriNet.slnx                # SLN replacement (novy .slnx format)
8 |-- TODO.md                      # Roadmap, zmeny, code-review log
9 |-- PetriEditor.Shared/          # Domenove modely a kontrakty
10 |-- PetriEditor.Analysis/        # Algoritmy formalni analyzy
11 |-- PetriEditor.Client/          # Blazor WASM klient
12 |-- PetriEditor.Server/          # ASP.NET Core host
13 '-- PetriEditor.Tests/           # xUnit testy (196)

```

4.1.2 Větvení a workflow

Aplikuje se zjednodušený **trunk-based** model:

- **master** — hlavní větev, vždy buildovatelná, vždy s procházejícími testy (32 commitů celkem v čase psaní dokumentu).
- **feature/*** — pracovní větve pro nové funkce (např. `feature/manual-graph-layout` obsahuje refaktor grafu z Cytoscape na vlastní canvas renderer).
- **archive/*** — archivované předchozí varianty (např. `archive/original-blazor-wasm` obsahuje původní single-project verzi před modularizací).

Slévání do **master** probíhá výhradně přes **pull request** (#1, #2 v historii) s manuální revizí změn. Velké refaktory jsou prováděny v sérii malých commitů s popisnými zprávami:

```

1 2fc6285 Refactor graph view: new layout, canvas, remove Cytoscape
2 cea84d9 Hide uncertain analyses for nets with inhibitor/reset arcs
3 c9e88d1 Fix DI scope, timer disposal, nullability, and error logging
4 8777e1d Auto-layout, chunked graph streaming, and test coverage
5 4d726b7 Add integration and regression tests for analysis engines

```

4.2 Build systém

Projekt používá nativní **.NET CLI** (`dotnet`) jako build systém. Žádné externí orchestrátory (Make, Cake, NUKE) se nepoužívají — výhodou je nulová nárazová křivka pro každého .NET vývojáře a přímá integrace s IDE.

4.2.1 Klíčové příkazy

Příkaz	Účel
<code>dotnet restore</code>	Stažení NuGet balíčků pro celé řešení.
<code>dotnet build</code>	Sestavení všech projektů (Debug).
<code>dotnet build -c Release</code>	Optimalizovaný release build.
<code>dotnet build /p:TreatWarningsAsErrors=true</code>	„Lint“ režim — varování blokuje build.
<code>dotnet test</code>	Spuštění 220 xUnit test. případů.
<code>dotnet test --collect:'XPlat Code Coverage'</code>	Testy + sběr code coverage.
<code>dotnet run --project PetriEditor.Server</code>	Lokální spuštění (dev. server).
<code>dotnet publish -c Release -o publish/</code>	Publikace pro nasazení (self-contained výstup).

4.2.2 NuGet závislosti

Externí balíčky jsou definovány v `.csproj` souborech (jeden centralní `Directory.Packages.props` se nepoužívá kvůli malému počtu závislostí). Klíčové balíčky:

Balíček	Verze	Použito v
Microsoft.AspNetCore.Components.WebAssembly	10.0.2	Client
Microsoft.AspNetCore.Components.WebAssembly.Server	10.0.2	Server
Microsoft.AspNetCore.SignalR.Client	9.0.4	Client
Microsoft.AspNetCore.DataProtection	9.0.3	Server
QuestPDF	2024.10.4	Server (PDF export)
Z.Blazor.Diagrams (+ Core)	3.0.4	Client
xunit	2.9.3	Tests
coverlet.collector	6.0.4	Tests
Microsoft.NET.Test.Sdk	17.14.1	Tests

4.3 Konfigurace aplikace

Aplikace pracuje s konfigurací na třech úrovních:

4.3.1 appsettings.json (server)

Serverový projekt používá standardní ASP.NET Core konfigurační hierarchii: `appsettings.json` (defaultní) → `appsettings.Development.json` (override) → environment variables (nejvyšší priorita).

```

1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*"
9 }

```

4.3.2 Environment variables

Pro nasazení na PaaS s ephemerním souborovým systémem (Railway, Heroku) program rozpoznává tyto proměnné prostředí:

Proměnná	Účel
ASPNETCORE_ENVIRONMENT	Development / Production (řídí WASM debugger, exception handler).
DP_KEYS_PATH	Cesta pro perzistenci DataProtection klíčů (default: <code>AppContext.BaseDirectory/dataprotection-keys</code>).
LOW_MEMORY_GC	Zapne <code>GC.LatencyMode.Batch</code> pro hostingy s omezenou pamětí.
RAILWAY_ENVIRONMENT	Auto-detekce Railway hostingu (alias pro <code>LOW_MEMORY_GC</code>).

4.3.3 Klientská konfigurace

Klient (Blazor WASM) má oddělený konfigurační kanál:

- `DiagramSettings` — runtime singleton držící vizuální a behaviorální parametry (velikost grid, barvy hran, velikosti uzlů). Je inicializován z `appsettings.json` přes `IConfiguration.Bind` a má event `Changed` pro re-render.
- `localStorage` (browser) — perzistentní uživatelské preference (firing delay simulate, zapnutí/vypnutí grid overlay). Není to server-side state — žádný cookies, žádná databáze.

4.4 Dockerizace

Projekt je distribuovatelný jako jediný Docker image. Dockerfile používá **multi-stage build**, aby výsledný image obsahoval pouze runtime artefakty bez SDK:

```

1 FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
2 WORKDIR /src
3 COPY . .
4 RUN dotnet test PetriEditor.Tests/PetriEditor.Tests.csproj \
5     -c Release --no-restore

```

```
6 RUN dotnet publish PetriEditor.Server/PetriEditor.Server.csproj \
7     -c Release -o /app/publish
8
9 FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
10 WORKDIR /app
11 COPY --from=build /app/publish .
12 ENV ASPNETCORE_ENVIRONMENT=Production
13 EXPOSE 8080
14 ENTRYPOINT ["dotnet", "PetriEditor.Server.dll"]
```

Klíčové body:

- **Testy jsou součástí buildu** (řádek 4): pokud `dotnet test` selže, image se nepostaví — bránění chybnému deploy.
- **Two-stage build**: výsledný image (`aspnet:10.0`) je menší než SDK image (`sdk:10.0`). Reálná velikost po publikaci ~200 MB.
- **Production environment fixed** (řádek 10) — žádný accidentální development režim.

4.5 CI/CD

V současné fázi projekt **nepoužívá automatizovanou CI/CD pipeline**. Adresář `.github/workflows/` je vytvořen, ale prázdný. Důvody:

- Projekt je jednočlenný (akademický kontext), takže nedochází k souběžnému psaní více vývojářů, kde CI brání rozbití `masteru`.
- **Dockerfile sám o sobě supluje CI** — testy jsou součástí buildu (řádek 4 Dockerfile), takže každé provedené nasazení implicitně garantuje zelené testy.
- Lokální workflow je jednoduchý: `dotnet test` → `git commit` → `git push` → PR review → merge.

Plán pro CI/CD Do připraveného adresáře `.github/workflows/` lze umístit GitHub Actions soubor `ci.yml` přibližně v této podobě:

```
1 name: CI
2 on:
3   push:
4     branches: [master]
5   pull_request:
6     branches: [master]
7 jobs:
8   build-test:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v4
12      - uses: actions/setup-dotnet@v4
13        with:
14          dotnet-version: 10.0.x
15      - run: dotnet restore
16      - run: dotnet build --no-restore -c Release
```

```
17 - run: dotnet test --no-build -c Release \
18     --collect:"XPlat Code Coverage"
19 - uses: codecov/codecov-action@v4
```

Tento workflow zajistí, že každý PR proti `master` bude před slevněním automaticky ověřen.

5 Bezpečnostní opatření a omezení

Přestože aplikace neukládá uživatelská data ani neimplementuje autentizaci (jde o bezstavovou nástrojovou aplikaci), na serverové straně jsou implementována standardní opatření proti zneužití veřejně exponovaného endpointu:

Antiforgery + DataProtection ASP.NET Core antiforgery tokeny jsou aktivní (`app.UseAntiforgery()`). Klíče `DataProtection` jsou perzistovány na disk (`DP_KEYS_PATH`), takže tokeny vystavené před restartem serveru zůstávají platné pro již aktivní klientské sessions.

Limit velikosti vstupní sítě `AnalysisHub.ValidateNetSize` odmítne sítě s více než 100 000 prvky (`MaxNetElements`) výjimkou `HubException` ještě před spuštěním analýzy. Brání DoS přes obří JSON payload.

Globální semafor + per-connection throttling `SemaphoreSlim` velikosti `max(2, ProcessorCount)` omezuje souběžný počet těžkých analytických operací na celém serveru. Na úrovni jednotlivého SignalR connection je nastaven 250 ms throttle, takže jeden klient nemůže zaplavit hub opakovanými voláními.

Wall-clock deadline 60 s Každá hub-metoda (`RunAnalysis`, `RunGraphAnalysis`, `ExportPdf`) má vlastní `CancellationTokenSource` s deadline 60 s. Po jeho vypršení je celý výpočet zrušen a klientovi se vrátí `HubException`.

SignalR receive size cap `MaximumReceiveMessageSize` je nastaveno na 10 MB. Větší zprávy jsou zahozeny dříve, než se začnou deserializovat.

Subresource Integrity Externí `Cytoscape.js` načítaný z CDN má `integrity="sha384-..."`, `crossorigin` a `referrerpolicy="no-referrer"`. Pokud by byl CDN obsah nahrazen, prohlížeč skript odmítne.

Známa omezení (mimo bezpečnost):

- Aplikace *neimplementuje persistenci* — diagramy se po přenačtení stránky ztratí. Plánovaná infrastrukturní vrstva (sekce v SAD) by tento bod adresovala.
- Není podpora *časovaných Petriho sítí*; priority přechodů jsou implementovány, ale nedeterministická volba při shodné prioritě je řešena pevným pořadím dle ID.
- *Coverage UI vrstvy je nízká* — Blazor komponenty nejsou pokryty automatizovanými testy (chybí `bUnit` dependency). Manuální E2E testování v Chrome a Firefox je dokumentováno v sekci 3.

6 Závěr

Petri Net Analyzer je implementován jako modulární Blazor Auto aplikace rozdělená do pěti .NET projektů. Jádrem aplikace (`PetriEditor.Analysis`) je čistá doménová knihovna bez vazby na UI ani web, což umožňuje její sdílení mezi klientem (WebAssembly) a serverem (ASP.NET Core) a zároveň úplné pokrytí jednotkovými testy.

Testovací strategie se opírá o 196 xUnit testovacích metod ve formě 220 testovacích případů (4045 řádků) rozdělených do pěti kategorií (jednotkové, theory, integrační, cross-engine, coverage-gap) — všechny procházejí v ~ 2 sekundách. Pokrytí algoritmického jádra `PetriEditor.Analysis` dosahuje **81.3 %** (line coverage), přičemž většina jednotlivých enginů má 95–100 %. Build a deploy je zajištěn nativním .NET CLI a multi-stage Docker imagem, který testy exekvuje jako součást build pipeline. Verzování probíhá v Gitu na GitHubu v trunk-based modelu s pull-request review.

Detailní popis jednotlivých návrhových rozhodnutí, sekvenčních diagramů a class modelu je obsažen v doprovodném *Software Architecture Documentu*.